

Attributing Wall-Clock Speedup in Draft Model Dimensions for Knowledge-Distilled Speculative Decoding

Yuchen Ouyang | 388166 | yuchen.ouyang@epfl.ch
 Zhiyan Ke | 407061 | zhiyan.ke@epfl.ch
 Tianyi Zhu | 386499 | tianyi.zhu@epfl.ch
 Siyuan Zhang | 388143 | siyuan.zhang@epfl.ch
 ShallowSeek

Abstract

Speculative decoding can reduce autoregressive inference cost by letting a small draft model propose tokens that a larger target model verifies. However, wall-clock speedup depends on more than acceptance rate. We study this relation through target-response generation, draft distillation, speculative decoding, and acceptance- and speedup-oriented evaluation. We compare Cross Entropy(CE), forward KL, reverse KL, and Jensen–Shannon divergence, and ablate KD temperature, data scale, response source, and KD-CE interpolation. With Qwen2.5-3B/0.5B, KD improves draft–target alignment but does not yield practical speedup. With a larger Qwen3-8B/0.6B target–draft gap and vLLM inference, speculative decoding reaches up to $1.19\times$ speedup under greedy decoding with $\gamma = 1$. Our results show that practical acceleration requires both an aligned draft model and suitable runtime parameters.

1 Introduction

Speculative decoding reduces the cost of autoregressive decoding by using a small draft model to propose tokens and a larger target model to verify them in parallel (Leviathan et al., 2023). Yet, acceptance rate alone does not determine acceleration. The draft must be aligned with the target, but it must also be cheap enough to offset the extra cost of drafting, verification, rejection handling, and KV-cache management. The speculative block size γ further changes this trade-off (Liu et al., 2024; Chen et al., 2024; Huang et al., 2024). Thus, better acceptance does not always produce proportional runtime gains.

Knowledge distillation (KD) can improve draft-target alignment. Recent work applies KD to speculative decoding and reports gains in acceptance or latency (Zhou et al., 2024; Xu et al., 2024; Hong et al., 2025; Zafrir et al., 2025). However, prior studies often combine several adaptation choices,

making it hard to isolate which factors affect wall-clock speedup.

We study both training and inference factors. First, we use Qwen2.5-3B-Instruct as the target and Qwen2.5-0.5B-Instruct as the draft to test how training choices affect alignment (Yang et al., 2024). Second, we evaluate runtime behavior with vLLM using a larger Qwen3-8B/Qwen3-0.6B pair, since wall-clock speedup also depends on system overhead and the target–draft cost gap.

Our results show that JSD gives the best overall draft quality in the Qwen2.5 training ablations. Pure KD ($\alpha = 1$), target-generated responses, larger training sets, and $T = 1$ also improve alignment. Yet the best Qwen2.5 setting remains slower than vanilla decoding. With Qwen3-8B/0.6B, speculative decoding reaches $1.19\times$ speedup under greedy decoding with $\gamma = 1$. These findings show that end-to-end acceleration depends jointly on distillation quality and runtime configuration.

2 Methodology

Our pipeline includes KD training of the draft model and speculative-decoding evaluation. We use UltraChat prompts and compare original dataset with target-generated responses. In the target-generated setting, responses are sampled from $p_\theta(\cdot | x)$ and used as draft training continuations. For token position t , let z_t^T and z_t^D denote target and draft logits. With KD temperature T ,

$$p_t = \text{softmax}(z_t^T/T), \quad q_t = \text{softmax}(z_t^D/T).$$

The draft is trained with

$$\mathcal{L} = \alpha \mathcal{L}_{\text{KD}} + (1 - \alpha) \mathcal{L}_{\text{CE}},$$

where $\alpha = 1$ is pure KD and $\alpha = 0.5$ mixes KD with CE. We compare three KD losses:

$$\mathcal{L}_{\text{FKL}} = D_{\text{KL}}(p_t \| q_t),$$

$$\mathcal{L}_{\text{RKL}} = D_{\text{KL}}(q_t \| p_t),$$

$$\mathcal{L}_{\text{JSD}} = \frac{1}{2}D_{\text{KL}}(p_t \| m_t) + \frac{1}{2}D_{\text{KL}}(q_t \| m_t),$$

where $m_t = (p_t + q_t)/2$. These represent a mode-covering objective (FKL), a mode-seeking objective (RKL), and a symmetric bounded objective (JSD) (Gu et al., 2024; Wen et al., 2023; Ko et al., 2024). At inference, the draft proposes γ tokens:

$$\hat{x}_{t+1:t+\gamma} \sim \prod_{i=1}^{\gamma} q_{\phi}(\cdot \mid x_{1:t}, \hat{x}_{t+1:t+i-1}).$$

The target verifies the block in parallel. Accepted tokens are appended; after rejection, the remaining draft suffix is discarded and generation resumes from the target-corrected token.

We report acceptance rate, average accepted tokens, and wall-clock speedup:

$$\text{Speedup} = T_{\text{vanilla}}/T_{\text{SD}}.$$

After training ablations, we select JSD with $T = 1$, $\alpha = 1$, target-generated responses, and 50k examples. We then run vLLM runtime ablations over γ , maximum generation length, and decoding mode.

3 Experiments

3.1 Data

We use the `train_sft` split of HuggingFaceH4/ultrachat_200k as the base instruction-following dataset (Ding et al., 2023). Each example is converted into a prompt-response pair (x, y) . We build 10k and 50k training subsets, with separate validation and evaluation sets. We use response-only supervision by masking prompt-token labels. We compare two response sources. In the *original* setting, y is the UltraChat assistant response. In the *target-generated* setting, prompts are fixed and responses are replaced by greedy continuations from the frozen Qwen2.5-3B-Instruct target, forming (x, y^T) .

3.2 Models and baselines

For KD ablations, we use Qwen2.5-3B-Instruct as the frozen target and Qwen2.5-0.5B-Instruct as the draft (Yang et al., 2024). The draft is initialized from the pretrained 0.5B model and fine-tuned with KD. This pair is less suitable for final wall-clock evaluation. In preliminary tests, distilled drafts improve acceptance metrics but remain slower than vanilla target-only decoding, on both HuggingFace-assisted speculative decoding, and vLLM speculative decoding. The likely cause is the small target-draft cost gap leaving limited room for improvement. We then shift to Qwen3-8B as the target and

Qwen3-0.6B as the draft. In this regime, vLLM speculative decoding can exceed vanilla decoding after runtime tuning. We use two baselines. Each distilled draft is compared with the corresponding pretrained draft without KD. Speculative decoding is also compared with vanilla target-only decoding on the same prompts and decoding configuration. Wall-clock speedup is always measured relative to the vanilla baseline.

3.3 Training ablations

We train draft models with CE, FKL, RKL, and JSD. We ablate KD temperature $T \in \{1, 2\}$ along with greedy mode, loss weight $\alpha \in \{0.5, 1\}$, data scale, and response source. Unless stated otherwise, training uses Qwen2.5-3B-Instruct as teacher and Qwen2.5-0.5B-Instruct as student. ¹

3.4 Runtime ablations

After selecting the best training configuration, we evaluate the final draft with vLLM. We vary the speculative block size γ , `max_new_tokens`, and decoding mode. These ablations test the trade-off between accepted tokens, wasted draft computation, and overhead amortization.

3.5 Evaluation metrics

Acceptance rate is the fraction of drafted tokens accepted by the target. Average accepted tokens is the mean number of accepted draft tokens per speculative step. Wall-clock speedup is $T_{\text{vanilla}}/T_{\text{SD}}$; values above 1 indicate faster inference than vanilla decoding.

4 Results and Analysis

4.1 KD improves draft-target alignment

Table 1 shows that some KD objectives improves draft quality over the pretrained draft. Under UltraChat-50k with $\alpha = 1$ and $T = 1$, RKL and JSD increase acceptance rate and average accepted tokens. RKL gives the highest acceptance rate, while JSD gives the best preliminary speedup.

Table 1: Effect of KD temperature on UltraChat-50k.

	$T = 1$			$T = 2$		
KD Loss	Acc.	Avg. Tok.	Speedup	Acc.	Avg. Tok.	Speedup
Pretrained	0.450	1.801	0.508	0.450	1.801	0.508
FKL	0.451	1.804	0.509	0.461	1.845	0.519
RKL	0.472	1.888	0.521	0.451	1.806	0.507
JSD	0.471	1.886	0.524	0.461	1.843	0.512

¹We use Qwen3-8B as target and Qwen3-0.6B as draft for runtime ablation. Section 4.5 explains this transition.

KD temperature affects objectives differently. Increasing T from 1 to 2 improves FKL but hurts RKL and JSD. This suggests that softer distributions help FKL’s mode-covering behavior, while RKL and JSD work better with sharper target distributions. Since speculative decoding rewards exact draft-target agreement, excessive smoothing can reduce acceptance.

4.2 Pure KD outperforms KD mixed with CE

Table 2 compares $\alpha = 1.0$ and $\alpha = 0.5$. For RKL and JSD, adding CE lowers acceptance rate and average accepted tokens. This suggests that fitting observed response tokens can conflict with matching the target distribution. Since the target verifies draft outputs, target alignment is more important than matching the original data distribution. We therefore use $\alpha = 1$ in the final configuration.

Table 2: Effect of KD loss weight α .

KD Loss	$\alpha = 1.0$			$\alpha = 0.5$		
	Acc.	Avg. Tok.	Speedup	Acc.	Avg. Tok.	Speedup
Pretrained	0.452	1.807	0.499	0.452	1.807	0.499
FKL	0.451	1.802	0.496	0.451	1.802	0.501
RKL	0.471	1.883	0.520	0.465	1.860	0.519
JSD	0.470	1.880	0.512	0.437	1.747	0.494

4.3 Larger training scale improves acceptance

Table 3 shows that increasing training data from 10k to 50k improves draft quality for all KD objectives. The gain is largest for RKL, whose acceptance rate rises from 0.463 to 0.472. This suggests that draft-target alignment benefits from broader coverage of target behavior. We use 50k examples for the final distilled draft.

Table 3: Effect of training-data scale on UltraChat.

KD Loss	50k			10k		
	Acc.	Avg. Tok.	Speedup	Acc.	Avg. Tok.	Speedup
Pretrained	0.450	1.801	0.508	0.450	1.801	0.508
FKL	0.451	1.804	0.509	0.447	1.788	0.509
RKL	0.472	1.888	0.521	0.463	1.851	0.508
JSD	0.471	1.886	0.524	0.469	1.876	0.524

4.4 Target-generated data improves draft training

Table 4 compares original and target-generated responses. For FKL and JSD, target-generated responses improve acceptance because they come directly from the target distribution. RKL behaves differently, which learns an overly sharp distribution from target-generated responses, making performance less stable at $T = 1.0$.

Table 4: Effect of response sources.

KD Loss	Original Data			Target-Generated Data		
	Acc.	Avg. Tok.	Speedup	Acc.	Avg. Tok.	Speedup
Pretrained	0.450	1.801	0.508	0.454	1.820	0.504
FKL	0.451	1.804	0.509	0.472	1.890	0.531
RKL	0.472	1.888	0.521	0.463	1.850	0.527
JSD	0.471	1.886	0.524	0.492	1.970	0.541

4.5 Larger target-draft cost gap improves runtime behavior

Although KD improves alignment for Qwen2.5-3B/0.5B, speculative decoding remains slower than vanilla decoding in our setting, evaluated on both HuggingFace-assisted decoding and vLLM decoding. This indicates that the main bottleneck is not only draft quality but also the small computational gap between target and draft. With a 3B target, each avoided target step saves limited computation, while speculative decoding still adds draft generation, target verification, rejection handling, and KV-cache overhead.

We therefore switch to Qwen3-8B/0.6B for runtime evaluation. The larger gap makes avoided target steps more valuable while keeping the draft cheap. Since the Qwen3 experiments show the same qualitative training trends, we use this pair for the runtime sweep and wall-clock analysis.²

4.6 Runtime ablation

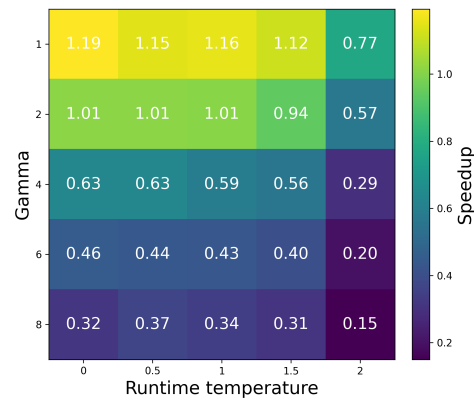


Figure 1: Runtime sweep over speculative block size γ and sampling temperature on Qwen3-8B/0.6B-JSD with max_new_tokens=256.

Speculative block size and sampling temperature Figure 1 shows that speedup depends strongly on γ and decoding temperature. The best

²We also experiment vllm speculative decoding with Qwen3-14B/Qwen3-0.6B pair, which reached 1.82x speedup. However, due to the VRAM limit, we were unable to train draft models with 14B model. But this result also suggested the effectiveness of model size gap bottleneck.

result, $1.19\times$, occurs with greedy decoding and $\gamma = 1$ with JSD trained draft. Speedup decreases as γ increases because longer draft blocks add cost and rejected suffixes waste computation.

Higher decoding temperature also reduces speedup by lowering draft-target agreement. At $\gamma = 1$, speedup drops from $1.19\times$ with greedy decoding to $0.77\times$ at temperature 2.0. For $\gamma \geq 4$, all settings fall below $1\times$ speedup.

These results show that runtime gains require both high acceptance and low draft overhead. For Qwen3-8B/0.6B, short speculative blocks and low-temperature decoding are most effective.

Generation length Table 5 reports the sweep over `max_new_tokens` under greedy decoding with $\gamma = 1$. Speedup is highest at 256 tokens, reaching $1.194\times$. The 128- and 512-token settings are similar, with $1.184\times$ and $1.186\times$. This suggests that overhead is already amortized at moderate generation lengths. Acceptance slightly decreases from 0.770 at 128 tokens to 0.704 at 512 tokens, but speedup remains stable.

Table 5: Runtime sweep over generation length.

<code>max_new_tokens</code>	Acceptance Rate	Tokens/s	Speedup
128	0.770	625.65	1.184
256	0.722	634.20	1.194
512	0.704	610.20	1.186

4.7 Overall analysis

Improving draft quality is necessary but not sufficient for wall-clock acceleration. KD improves acceptance metrics, especially with JSD, pure KD, target-generated data, and larger training sets. However, speedup also depends on target-draft cost gap, decoding parameters, and system overhead. The Qwen2.5-3B/0.5B pair is too small to offset speculative overhead, while Qwen3-8B/0.6B reaches practical speedup after vLLM runtime tuning. In this setting, smaller γ and lower temperature improve speedup, while generation length has a smaller effect.

5 Ethical considerations

This project studies KD for speculative decoding as an inference-efficiency method. The draft model is trained to approximate the target model and may inherit its biases, hallucinations, and unsafe behaviors. UltraChat responses and target-generated data may also contain social biases or model-specific

artifacts. Therefore, our method should not be viewed as improving factuality, fairness, or safety.

Language coverage is another concern. The method may work better for high-resource languages than for low-resource languages such as Urdu or Swahili, where weaker draft-target alignment can reduce acceptance, speedup, or output quality. These gaps may disproportionately affect underrepresented language communities.

Lower generation cost may also enable misuse, including misinformation, spam, phishing, or biased automated responses at scale. Responsible deployment should keep target-model safety monitoring, evaluate toxicity, bias, factuality, and quality across languages, and add language-specific distillation data when needed.

6 Conclusion

This project studied how draft-model distillation and runtime configuration affect speculative decoding. In the KD ablations, FKL as well as JSD provides the strongest overall performance, in terms of acceptance and preliminary speedup. The best settings use greedy decoding, pure KD ($\alpha = 1$), target-generated responses, and 50k training examples. These choices improve draft-target alignment more consistently than higher temperature, CE interpolation, original responses, or smaller training sets.

However, improved alignment alone does not guarantee wall-clock speedup. With Qwen2.5-3B/0.5B, speculative decoding remains slower than vanilla decoding even after distillation. This shows that system-level factors matter. With a larger Qwen3-8B/0.6B target-draft gap and vLLM inference, speculative decoding becomes beneficial. Runtime sweeps show that short speculative blocks, low-temperature decoding, and moderate generation lengths give the best results.

Our best configuration combines JSD, $\alpha = 1$, target-generated responses, and 50k examples for training, with greedy decoding and $\gamma = 1$ at inference time. It reaches up to $1.19\times$ wall-clock speedup. The main conclusion is that practical speculative decoding requires both strong draft-target alignment and a runtime regime where accepted draft tokens outweigh speculative overhead.

References

- Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. 2024. Sequoia: Scalable, robust, and hardware-aware speculative decoding. *arXiv preprint arXiv:2402.12374*.
- Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2023. [Enhancing chat language models by scaling high-quality instructional conversations](#).
- Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2024. Minillm: Knowledge distillation of large language models. In *The twelfth international conference on learning representations*.
- Fenglu Hong, Ravi Raju, Jonathan Lingjie Li, Bo Li, Urmish Thakker, Avinash Ravichandran, Swayambhoo Jain, and Changran Hu. 2025. Training domain draft models for speculative decoding: Best practices and insights. *arXiv preprint arXiv:2503.07807*.
- Kaixuan Huang, Xudong Guo, and Mengdi Wang. 2024. Specdec++: Boosting speculative decoding via adaptive candidate lengths. *arXiv preprint arXiv:2405.19715*.
- Jongwoo Ko, Sungnyun Kim, Tianyi Chen, and Se-Young Yun. 2024. Distillm: Towards streamlined distillation for large language models. *arXiv preprint arXiv:2402.03898*.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. [Fast inference from transformers via speculative decoding](#).
- Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. 2024. [Online speculative decoding](#). In *Proceedings of the 41st International Conference on Machine Learning, ICML'24*. JMLR.org.
- Yuqiao Wen, Zichao Li, Wenyu Du, and Lili Mou. 2023. F-divergence minimization for sequence-level knowledge distillation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10817–10834.
- Wenda Xu, Rujun Han, Zifeng Wang, Long T Le, Dhruv Madeka, Lei Li, William Yang Wang, Rishabh Agarwal, Chen-Yu Lee, and Tomas Pfister. 2024. Speculative knowledge distillation: Bridging the teacher-student gap through interleaved sampling. *arXiv preprint arXiv:2410.11325*.
- An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. 2024. [Qwen2 technical report](#).
- Ofir Zafrir, Igor Margulis, Dorin Shteyman, Shira Guskina, and Guy Boudoukh. 2025. Fastdraft: How to train your draft. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22488–22505.
- Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. 2024. [Distillspec: Improving speculative decoding via knowledge distillation](#).